

Reproduction and Critical Study of “HTTP2vec: Embedding of HTTP Requests for Detection of Anomalous Traffic”

Data Science in Cyber — Final Project
Course lecturer: Dr. Uri Itai

July 4, 2026

Abstract

This project reproduces and critically evaluates the paper *HTTP2vec: Embedding of HTTP Requests for Detection of Anomalous Traffic* (Gniewkowski et al., 2021, arXiv:2108.01763). The method treats a raw HTTP request as text, learns a RoBERTa masked-language model on *normal* traffic only, embeds each request as the mean of its last hidden layers, and classifies the resulting vectors. We re-implement the full pipeline from scratch, run it on the CSIC 2010 dataset (a seeded 40% subset, 5 epochs), and compare seven classifiers, a trainable MLP head, and—new in this version—two *non-embedding* baselines (hand-crafted descriptive features and a character n -gram TF-IDF model). We reproduce the paper’s *qualitative* central claim—RoBERTa embeddings make normal and anomalous HTTP requests strongly separable for supervised classifiers (best $F1 = 0.913$, $MCC = 0.853$, $ROC-AUC = 0.984$)—but we do *not* reproduce its most impressive headline numbers (near-zero false-positive rates, near-perfect scores on CSE-CIC-IDS2018), and we show with our own experiments that the paper’s optimism about *unsupervised* detection on the same embeddings is not supported (Isolation Forest $F1 = 0.47$, LOF $F1 = 0.34$). Most tellingly, a trivial character n -gram TF-IDF + linear SVC baseline *beats* the RoBERTa embedding on every metric ($F1 = 0.975$ vs 0.913 , $FPR_{99} = 0.034$ vs 0.210), and even a hand-crafted Random Forest matches it—so on CSIC 2010 the expensive contextual representation is not just non-SOTA, it is not even necessary. We further show that request-line templating leaks across the train/test split (77.7%) and that removing it collapses the best model’s FPR_{99} from 0.21 to 0.80. We argue that several of the paper’s strongest results are driven by dataset artifacts and an unrealistic class base rate rather than by the representation itself.

Contents

1	Summary of the Source	3
1.1	The problem being addressed	3
1.2	Why the problem is important	3
1.3	The proposed solution	3
1.4	The dataset used	3
1.5	The methodology employed	3
2	Critical Evaluation of the Author’s Claims	4
2.1	Claim 1 — RoBERTa embeddings give state-of-the-art supervised detection . . .	4
2.2	Claim 2 — The representation is learned “unsupervised” and suits real, label-free deployments	4
2.3	Claim 3 — The classes are “well separated” and form clean clusters in 2D	4
2.4	Claim 4 — Near-perfect results on CSE-CIC-IDS2018 demonstrate the method .	4
2.5	Claim 5 — The method is interpretable	5
2.6	Is the evaluation methodology appropriate?	5
2.7	Are the conclusions justified?	5

3	Exploratory Data Analysis (EDA)	5
3.1	Data loading, inspection and quality	6
3.2	Class balance (prevalence) and its real-world meaning	6
3.3	Feature distributions by class	6
3.4	Correlation analysis and choice of coefficient	7
3.5	Redundancy spotted during EDA	8
3.6	Group-by / crosstab: an accidental label proxy	9
3.7	Outliers and the (absent) temporal dimension	9
4	Feature Engineering Analysis	9
4.1	Was feature engineering performed, and on which features?	9
4.2	Transformations applied, why, and their effect	9
4.3	Is there redundancy, how would you spot it, and how would you tackle it?	10
4.4	Was the feature engineering meaningful? Mathematical and security intuition	11
4.5	Additional features that could improve performance	11
5	Reproducibility Analysis	11
5.1	Can the code be executed?	11
5.2	Are all required files and dependencies available?	12
5.3	Are there hidden preprocessing steps?	12
5.4	Overall reproducibility	12
6	Experimental Results	12
6.1	Experimental setup and the modifications we introduced	12
6.2	The RoBERTa language model	13
6.3	Evaluation metrics: definitions and why they matter here	14
6.4	Supervised classifiers and the unified comparison	14
6.5	Do we even need the embedding? Hand-crafted and lexical baselines	15
6.6	Ranking quality and the best model	17
6.7	The trainable MLP head	18
6.8	Reproduction vs the paper	18
6.9	The cost of templating: leakage and de-duplicated re-evaluation	19
7	Error Analysis	19
7.1	Examples of failures — unsupervised Isolation Forest	20
7.2	Examples of failures — best supervised model (Logistic Regression)	20
7.3	Patterns, implications and the FP/FN trade-off	21
8	Conclusions	21
8.1	Claim verdicts at a glance	21
8.2	Key findings	22
8.3	Lessons learned	22
8.4	Strengths and weaknesses of the proposed solution	22
8.5	Suggestions for future improvements	23
9	Executive Summary	23
10	Summing It Up	24

Reproduced source. M. Gniewkowski, H. Maciejewski, T. Surmacz, W. Walentynowicz, *HTTP2vec: Embedding of HTTP Requests for Detection of Anomalous Traffic*, arXiv:2108.01763v1 (2021).

Reproduction repository. <https://github.com/nirmanor1/cyberAno>

Dataset. CSIC 2010 HTTP dataset (Spanish Research National Council, CSIC), <https://www.tic.itefi.csic.es/dataset/>

1 Summary of the Source

1.1 The problem being addressed

HTTP is the transport used by the large majority of web traffic, and therefore by the majority of web attacks: SQL injection, cross-site scripting (XSS), CRLF injection, buffer overflows, information gathering, and file disclosure all travel inside HTTP requests. The paper addresses *web attack / anomaly detection*: given a raw HTTP request, decide whether it is legitimate (*normal*) or an attack (*anomaly*). This is a Web Application Firewall / Intrusion Detection System (IDS) problem.

1.2 Why the problem is important

According to the OWASP Top Ten, injection is a top web threat. Classic defences are either *signature/rule-based* (e.g. Snort) or trained on *hand-crafted features*. Rule-based systems require constant expert maintenance and miss novel attacks; hand-crafted features are brittle and encode analyst bias. An accurate, low-maintenance detector that learns the structure of traffic directly would reduce both missed attacks (breaches) and false alarms (analyst fatigue), and, if *interpretable*, would also help post-incident analysis.

1.3 The proposed solution

The core idea is to treat an HTTP request as a sentence in an (artificial) language and borrow modern NLP representation learning. Concretely, the authors:

1. train a **byte-level BPE (BBPE) tokenizer** on all traffic, so there is no out-of-vocabulary problem for adversarial payloads;
2. train a **RoBERTa** masked-language model (MLM) *only on normal traffic*, so it learns what “normal” HTTP looks like;
3. **embed** each request as the mean over its lines of the concatenation of the last four hidden layers (a $4 \times 768 = 3072$ -dimensional vector);
4. **classify** the embeddings with standard scikit-learn models: Logistic Regression (LR), Random Forest (RF) and linear Support Vector Classification (SVC).

Two further contributions are emphasised: (i) **interpretability**, via a token-removal procedure that scores how much each token pushes a request across the classifier’s decision boundary; and (ii) **visualisation / clustering** of the embedding space with t-SNE.

1.4 The dataset used

The paper evaluates three datasets: **CSIC 2010** (the canonical text-form HTTP attack dataset), **CSE-CIC-IDS2018** (extracting web attacks from packet captures) and **UMP** (a dataset the authors collected themselves). Each dataset is split into a *training* part (normal only, used to fit RoBERTa) and an *inference* part (normal + anomalous, used for classification). For CSIC 2010 the paper uses 36,000 normal training requests, 36,000 normal inference requests and 25,065 anomalous inference requests (Table 1 of the paper). **Our reproduction focuses on CSIC 2010**, which is the only one of the three available in a ready-to-use text form.

1.5 The methodology employed

The RoBERTa model is the “base”-style configuration but shrunk to 6 layers, hidden size 768, 12 attention heads, maximum sequence length 512, trained for 10 epochs with batch size 32 and 15% masked tokens. Evaluation follows a **stratified 5-fold cross-validation**, reporting **F1**, **MCC**, and **FPR@TPR** (FPR₉₀ and FPR₉₉: the false-positive rate required to catch 90% / 99% of attacks). The headline CSIC 2010 result is F1 = 0.969 (SVC) with FPR₉₉ = 4.2%, and a “perfect” classifier (F1 = 0.999) on CSE-CIC-IDS2018.

2 Critical Evaluation of the Author’s Claims

This section is the heart of the project. We enumerate the paper’s main claims and test each against the evidence in the paper and against our own reproduction (Sections 6–7). A compact verdict table is given at the end (Table 7).

2.1 Claim 1 — RoBERTa embeddings give state-of-the-art supervised detection

Supported qualitatively; the “state-of-the-art” framing is not. Our reproduction confirms the *direction* of this claim: even on a 40% subset trained for only 5 epochs, simple classifiers on the embeddings reach $F1 = 0.91$, $MCC = 0.85$ and $ROC-AUC = 0.98$ (Table 3). The linear SVC working well tells us the classes are close to *linearly separable* in the 3072-dimensional space—exactly the paper’s argument. However, “SOTA” does not survive scrutiny on two fronts. First, the paper’s specific CSIC 2010 numbers are noticeably better than ours: SVC $F1 = 0.969$ vs our 0.910, and, more importantly, $FPR_{99} = 4.2\%$ vs our $\approx 21\%$; the gap on the low-false-positive metrics—the ones that matter operationally—is large (Section 6.8). Second, and more damaging, we added the obvious controls the paper omits: a *character n -gram TF-IDF* model and a *hand-crafted-feature* model (Section 6.5, Table 4). The lexical SVC beats the RoBERTa embedding on *every* metric ($F1 = 0.975$, $MCC = 0.958$, $FPR_{99} = 0.034$), and a hand-crafted Random Forest matches it ($F1 = 0.915$, $FPR_{99} = 0.125$). If a bag of character n -grams outperforms the transformer on this dataset, the embedding is not state-of-the-art here—it is strictly dominated by a baseline that costs seconds to train. Calling this “state-of-the-art” is further weakened by the leakage issues discussed under Claim 4.

2.2 Claim 2 — The representation is learned “unsupervised” and suits real, label-free deployments

Misleading as stated. The *representation* (RoBERTa) is indeed learned without labels—on normal traffic only—but the *detection* in the paper is fully *supervised*: LR/RF/SVC are trained on labelled normal + anomalous embeddings. The paper’s conclusion further speculates that *unsupervised* detection “is likely to be effective” on these embeddings. We tested this directly by fitting an Isolation Forest and a Local Outlier Factor detector on normal-only embeddings and scoring the mixed set. Both are far weaker than the supervised models: Isolation Forest reaches only $F1 = 0.47$ ($ROC-AUC = 0.75$) and LOF $F1 = 0.34$ (Table 3). So on our evidence the embeddings are excellent *features for a supervised classifier* but do *not* yet support effective unsupervised detection; the paper’s optimistic framing is not substantiated.

2.3 Claim 3 — The classes are “well separated” and form clean clusters in 2D

Only partially supported. The paper shows t-SNE plots and states that classes are “well separated even in a lower space”. Our t-SNE (Figure 7) does show clear *cluster structure*—requests group by endpoint/type—and one large, almost purely normal cluster. But many peripheral clusters are visibly *mixed* (red and blue together). The strong separability the classifiers exploit lives in the full 3072-dimensional space, not in the 2D projection; the visualisation somewhat oversells the “well separated” narrative. t-SNE is also a non-linear, perplexity-dependent embedding and should not be read as evidence of linear separability.

2.4 Claim 4 — Near-perfect results on CSE-CIC-IDS2018 demonstrate the method

This is essentially a data artifact, and the authors say so. The paper reports $F1 = 0.999$ on IDS2018 and, commendably, investigates *why*: the interpretability analysis reveals that the

tokens `Accept` and `Accept-Encoding` are “highly related to the anomaly subset” because those header lines appear in nearly unchanged form in almost every attack. That is a textbook *spurious correlation* / *data leakage*: the model learns an artifact of how the attack traffic was generated (a scanner with fixed headers), not a property of attacks. The same paper notes an analogous artifact in CSIC 2010—every anomalous request ends with `\n` rather than `\r\n`. A “perfect” score built on such artifacts is not evidence of generalisation, and the paper’s own conclusion (“the dataset is really simple to classify or we made some mistakes”) implicitly concedes this. We did not reproduce IDS2018 (it is not in ready text form), but we flag that this headline number should not be read as method quality.

2.5 Claim 5 — The method is interpretable

Plausible and a genuine strength, but not reproduced here. The token-removal attribution is a reasonable, model-agnostic explanation technique and is one of the paper’s more original contributions; the examples shown (SQL keywords such as `DROP/SELECT/FROM` dominating an SQL-injection request) are convincing. We deliberately kept this out of scope—our reproduction targets the *classification* claims—so we neither confirm nor refute the exact attribution scores. We note only that the technique’s cost (re-embedding one document per token) is high for long requests.

2.6 Is the evaluation methodology appropriate?

Mostly yes, with important caveats. **Good:** stratified k -fold CV; the correct choice of threshold-aware metrics (F1, MCC, FPR@TPR) instead of bare accuracy; and, crucially, the separation of the *normal-training* set (for the LM) from the *normal-inference* set (for classification), which we verified in the code and which avoids the most obvious form of leakage. **Caveats:** (i) the reported class base rate ($\approx 41\%$ anomalies) is wildly higher than any real deployment, which inflates precision, F1 and accuracy relative to production (Section 3.2); (ii) CSIC 2010 contains heavy near-duplication and method-level giveaways (Section 3.1) that make the task easier than real traffic; and (iii) the CAE baseline the paper compares against “converged to a local minimum ... and did not learn”, so the comparison to prior work is partly against a model the authors could not make work.

2.7 Are the conclusions justified?

The narrow conclusion—*RoBERTa embeddings of HTTP requests are strong features for supervised web-attack detection*—is justified and we reproduce it. The broader conclusions—SOTA performance, effectiveness of *unsupervised* detection, and clean 2D separability—are only partially justified: they are inflated by dataset artifacts, an unrealistic base rate, and (for the unsupervised claim) contradicted by our experiments. The paper is unusually honest about its own artifacts, which is to its credit, but the abstract and conclusion do not carry those caveats forward.

3 Exploratory Data Analysis (EDA)

All EDA is performed on a *descriptive* feature frame of the inference set: one row per request, 24,426 rows $\times$ 16 columns of human-readable, model-agnostic attributes (lengths, counts, ratios, entropy and two attack-signature flags) plus the label. These descriptive features exist *only* for EDA and error analysis—they are not the features the model uses (those are the RoBERTa embeddings, Section 4).

3.1 Data loading, inspection and quality

The loader returns three views matching the paper: `lm_train` (normal only, 14,400), `inference` (normal + anomaly, 24,426) and a tokenizer corpus (all traffic, 38,826). We verified in code that the normal inference requests come from a *different* file (`normalTrafficTest.txt`) than the LM-training normals (`normalTrafficTraining.txt`), so there is no train/inference leakage on the normal side. Data-quality checks returned no missing values, no constant columns and no exactly duplicated feature columns.

How duplicated is CSIC, really? Three levels of templating. Duplication is the single most important property of this dataset, but *where* it lives changes the story, so we measure it at three levels (Table 1):

Duplication level	Unique of 24,426	Duplicate rate
Full raw request text (headers + body)	24,426	0.0%
Request first line (method + URL template)	6,040	75.3%
Descriptive fingerprint (16 features)	8,748	64.2%

Table 1: Duplication at three granularities on the inference set. No two requests are byte-identical, yet three quarters reuse the same request-line template.

The contrast is the point. At the *raw-text* level nothing is duplicated (volatile fields—cookies, lengths, payload values—make every request unique), which is exactly why a naive exact-duplicate check finds nothing and would badly mislead. But at the *request-line* level 75.3% of requests reuse one of only 6,040 URL templates, and 64.2% collapse to the same 16-feature descriptive fingerprint. CSIC 2010 is generated by exercising a single e-commerce application, so requests are highly templated. Practically this means the effective diversity of the data is far lower than 24,426, cross-validation folds and the train/test split share near-identical templates (we quantify this leakage at 77.7% and its cost in Section 6.9), and reported performance is optimistic relative to heterogeneous real traffic. The columns and the default positional index are sensible: CSIC has no natural key (no request id, no timestamp), so a positional index that aligns row i with request i is the right choice and is what we use for error analysis (Section 7).

3.2 Class balance (prevalence) and its real-world meaning

The inference set is 14,400 normal (59%) and 10,026 anomaly (41%). This is only mildly imbalanced, so *accuracy* is not catastrophically misleading here—but it is deeply **unrealistic**. In production, web attacks are a tiny fraction of traffic (often $\ll 1\%$). At a realistic base rate, a detector with our reproduced $\text{FPR}_{99} \approx 21\%$ would drown analysts in false positives: to catch 99% of attacks it would misflag more than a fifth of *all* benign traffic, and because benign traffic dominates, precision would collapse far below the 0.92 we measure at 41% prevalence. The authors do not correct for this base-rate mismatch. Consequently we de-emphasise accuracy and prefer MCC, ROC-AUC and FPR@TPR, which are either imbalance-robust or explicitly operational (Section 6.3).

3.3 Feature distributions by class

Figure 1 shows six representative features split by class. `target_length`, `body_length` and `pct_encoding_ratio` are strongly right-skewed: most requests are short with little percent-encoding, but a heavy anomaly tail extends to long, heavily encoded payloads. `shannon_entropy` is the cleanest single-feature separator—anomalies shift towards higher character entropy—and `n_query_params` shows a discrete anomaly spike at high parameter counts. These are exactly the shapes one expects from injection/obfuscation payloads.

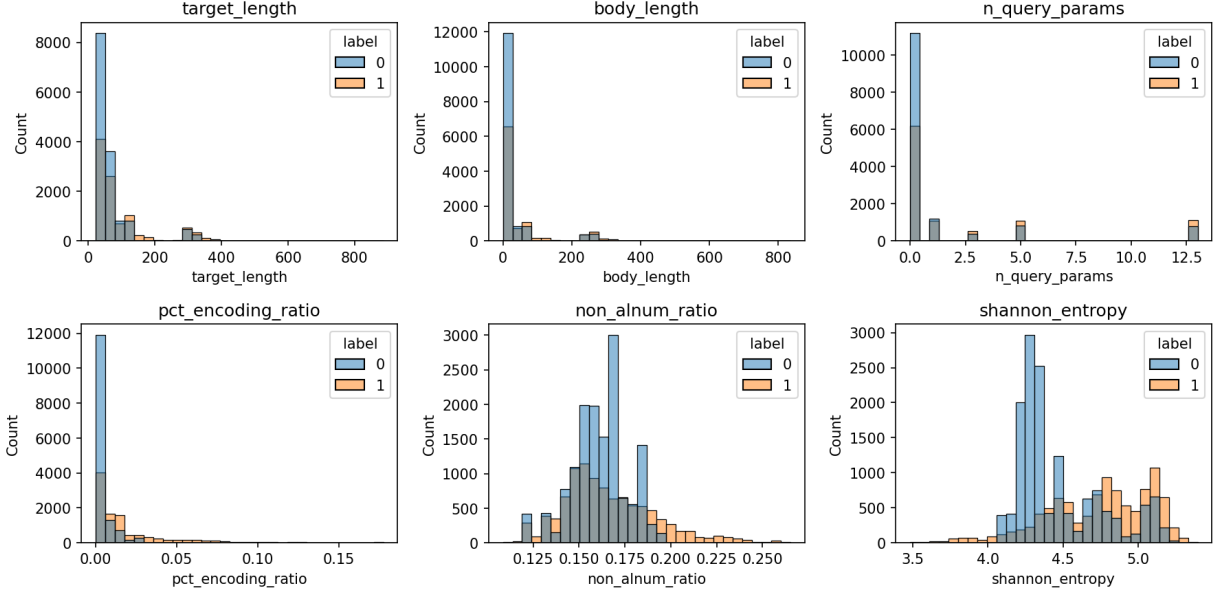


Figure 1: Distributions of six descriptive features by class (0 = normal, 1 = anomaly). Lengths and percent-encoding are heavy-tailed; entropy and query-parameter count show the clearest class shift.

3.4 Correlation analysis and choice of coefficient

We report **Spearman** rank correlation (Figure 2) as the primary measure. The three candidates studied in the course capture different relationships:

- **Pearson** measures *linear* association between continuous variables; it assumes roughly linear, normal-ish data and is sensitive to outliers.
- **Spearman** is the Pearson correlation of the *ranks*; it captures any *monotonic* relationship and is robust to outliers and non-normality.
- **Kendall** (τ) is also rank-based, counting concordant vs discordant pairs; it is preferred for small samples or many ties and is more conservative.

Our descriptive features are heavily skewed, non-normal and full of legitimate outliers (long attack payloads), and we care about *monotonic* “more of this $\Rightarrow$ more likely an attack” relationships rather than strict linearity. Spearman is therefore the appropriate choice; Pearson would be distorted by the heavy tails and Kendall is unnecessary given the large n .

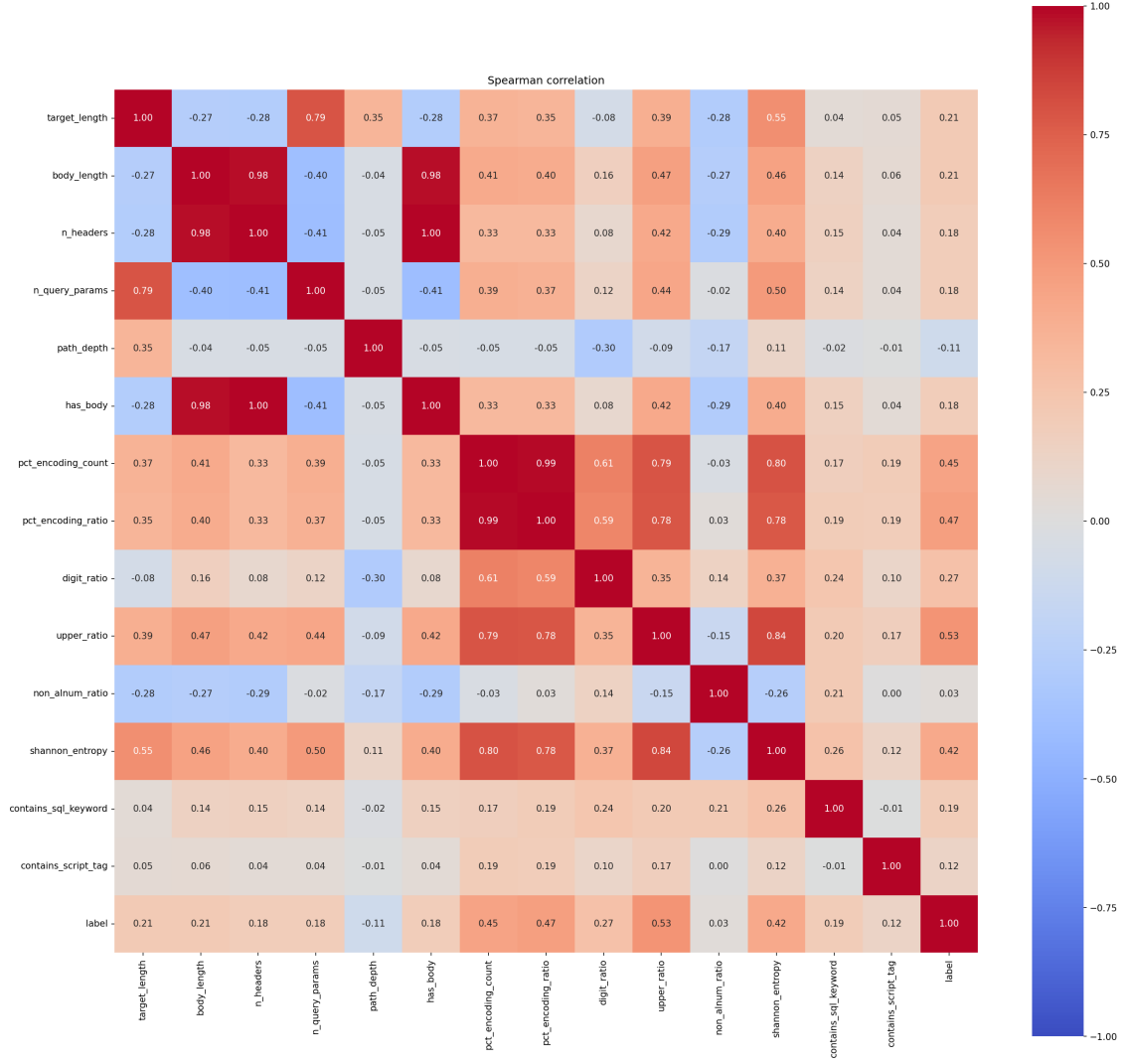


Figure 2: Spearman correlation heatmap of the descriptive features and the label. Note the near-perfectly collinear block $\{\text{body_length}, \text{n_headers}, \text{has_body}\}$ and the pair $\{\text{pct_encoding_count}, \text{pct_encoding_ratio}\}$.

Association with the label. The descriptive features most correlated with “anomaly” are `upper_ratio` ($\rho = 0.53$), `pct_encoding_ratio` ($\rho = 0.47$), `shannon_entropy` ($\rho = 0.42$) and `digit_ratio` ($\rho = 0.27$). These are *practically*, not merely statistically, significant: a ρ of 0.5 on a single cheap feature is a meaningful monotonic signal. The cybersecurity reading is direct—SQL keywords (DROP, SELECT, UNION) and XSS tags inject uppercase letters and unusual symbols, evasion relies on percent-encoding, and both raise character entropy. Even the crudest features carry attack signal—and, as Section 6.5 shows, they carry *enough* signal that a character n -gram model actually *outperforms* the contextual embedding on CSIC, a strong hint that this dataset is decided almost entirely by such surface patterns.

3.5 Redundancy spotted during EDA

The heatmap immediately exposes redundancy (revisited in Section 4.3): `body_length`, `n_headers` and `has_body` are almost perfectly collinear ($\rho = 0.98\text{--}1.00$), because in CSIC a request has a body iff it is a POST, which also carries the extra headers; `pct_encoding_count` and `pct_encoding_ratio` are $\rho = 0.99$; and `upper_ratio` tracks `shannon_entropy` at $\rho = 0.84$.

3.6 Group-by / crosstab: an accidental label proxy

Cross-tabulating the HTTP method against the label reveals a dataset artifact:

Method	Normal	Anomaly	Anomaly share
GET	11,119	6,040	35%
POST	3,281	3,818	54%
PUT	0	168	100%

Table 2: HTTP method vs label on the inference set. PUT appears *only* in anomalies—a partial label giveaway present in the raw data.

PUT occurs in 168 requests, *all* of them anomalous. A model can therefore “cheat” on those requests using the verb alone. This is another reason the task is easier on CSIC than in the wild and reinforces the artifact theme from Claim 4.

3.7 Outliers and the (absent) temporal dimension

`target_length` has median 50 but a 99th percentile of 340 and a maximum of 886; `body_length` is similar. These extreme values are overwhelmingly attack payloads, so here *outliers are signal, not noise*—we keep them rather than winsorising. Finally, CSIC 2010 has **no timestamps**, so a temporal / concept-drift analysis is impossible. This is a real limitation: production IDS must cope with drift (new endpoints, changing user behaviour), and neither the dataset nor the paper can speak to it. This aligns with world knowledge that attack and traffic mixes change over time.

4 Feature Engineering Analysis

4.1 Was feature engineering performed, and on which features?

Yes—but the paper’s thesis is precisely that the feature engineering should be *learned*, not hand-crafted. There are two distinct feature sets in this project:

1. **The model features (the paper’s contribution):** the 3072-dimensional RoBERTa embedding of each request. This is the only input the classifiers see.
2. **The descriptive features (our EDA add-on):** the 15 human-readable attributes of Section 3. They are never fed to the model; they exist to make the data inspectable and the errors explainable.

4.2 Transformations applied, why, and their effect

Encoding (text $\rightarrow$ numeric): byte-level BPE. Raw requests are strings, so an encoding step is unavoidable. The paper uses a *byte-level* BPE tokenizer trained on all traffic. *Why:* operating on bytes eliminates the out-of-vocabulary problem—percent-encoded bytes, random tokens in payloads and rare characters all decompose into known sub-tokens—while keeping the vocabulary small (30,000). Training the tokenizer on *all* traffic (not just normal) is a deliberate choice so that short/rare tokens are not themselves treated as anomalous. *Effect:* a robust, lossless encoding of adversarial text, which is the precondition for everything downstream.

Feature creation: contextual embeddings. Each request line is passed through RoBERTa; the last four hidden layers are concatenated (4×768), mean-pooled over tokens, and the per-line vectors are averaged into one request vector. *Why the last four layers:* different transformer layers encode different levels of abstraction, and concatenating the top layers is a well-known feature-based BERT recipe that empirically beats using the final layer alone. *Why mean pooling*

/ *line averaging*: it yields a fixed-length vector regardless of request length and is order- and length-robust. *Effect*: the resulting space is close to linearly separable (the linear SVC scores $F1 = 0.91$), which is the central empirical payoff.

Scaling: standardisation. Embedding dimensions are standardised (`StandardScaler`, zero mean/unit variance) inside the linear models, the anomaly detectors and the MLP head; the tree ensembles (Random Forest, gradient boosting) skip scaling because they are scale-invariant. *Why*: logistic regression, linear SVC, KNN and the distance/density-based detectors are all sensitive to feature scale; unscaled dimensions with large variance would dominate. *Effect*: required for the linear and distance-based models to behave; a pure convenience for trees, hence omitted there.

Dimensionality reduction: used for analysis, not for classification. The paper keeps the full 3072-dimensional vector for classification and explicitly lists its size as a limitation (“dimensionality ... ca. 3000 ... affects training and inference time”). We make this concrete with PCA (Figure 3): the first principal component already explains $\approx 60\%$ of the variance, and only $\approx 13\text{--}14$ components are needed to reach 90%. In other words the intrinsic dimensionality is an order of magnitude smaller than 3072—the representation is *highly redundant*. t-SNE (Figure 7) is used only for 2D visualisation, never as a classifier input.

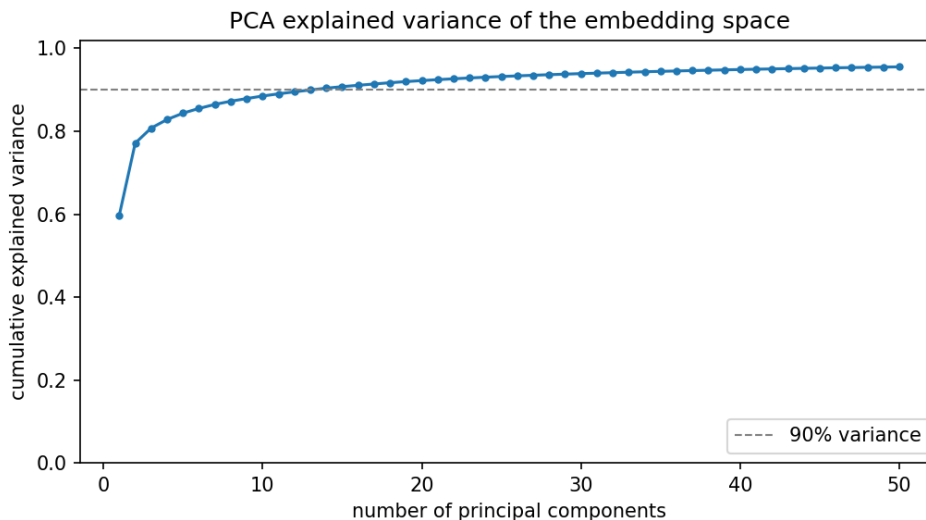


Figure 3: Cumulative PCA explained variance of the 3072-dimensional embedding space. About 13–14 components already capture 90% of the variance, quantifying the paper’s “too large a representation” limitation.

4.3 Is there redundancy, how would you spot it, and how would you tackle it?

Redundancy is present at both feature levels, and we used four complementary tools to spot it:

1. **Constant-column check** (variance = 0): none here, but the cheapest first screen.
2. **Exact duplicate-column check** (compare columns as vectors): none exactly identical.
3. **Correlation heatmap** (Section 3.4): exposes the collinear blocks $\{\text{body_length}, \text{n_headers}, \text{has_body}\}$ ($\rho \approx 1$) and $\{\text{pct_encoding_count}, \text{pct_encoding_ratio}\}$ ($\rho = 0.99$).
4. **PCA** (Figure 3): quantifies redundancy in the *embedding* space, where per-dimension correlation is not human-readable.

How to tackle it. For the descriptive features: drop all but one member of each collinear group, or replace the group with a single principal component. For the embedding: apply PCA

/ feature agglomeration to compress $3072 \rightarrow \sim 50$ dimensions with negligible loss (directly addressing the paper’s stated cost limitation), and/or rely on L_1/L_2 regularisation, which the linear models already provide, to damp redundant directions. Tree ensembles are inherently tolerant of correlated inputs. Reducing the dimensionality would also speed up the KNN, Isolation Forest and LOF detectors, whose distance computations degrade in high dimensions (the “curse of dimensionality”).

4.4 Was the feature engineering meaningful? Mathematical and security intuition

Mathematically: self-attention lets RoBERTa encode each token *in context*, so the same symbol (the paper’s example is “/”) maps to different vectors depending on its neighbours. Averaging contextual token vectors yields a smooth request representation in which semantically similar requests are near one another under Euclidean distance; a hyperplane can then separate “normal-shaped” from “attack-shaped” regions, which is why a *linear* classifier suffices. **From a security standpoint:** the descriptive analysis showed that attacks differ from normal traffic in uppercase ratio, percent-encoding and entropy—all local, token-level phenomena that a contextual language model captures far more richly than any fixed feature. The embedding effectively learns a data-driven superset of the hand-crafted signatures an analyst would otherwise maintain by hand. So yes, the feature engineering is meaningful: it converts an intractable string-matching problem into a well-behaved geometry problem.

4.5 Additional features that could improve performance

- **Response-side signals** (status code, response size, latency) from server access/error logs—the paper itself flags this as future work; a 500/302 pattern is a strong post-hoc attack signal.
- **Structural / template features:** URL path template (endpoint identity), parameter-name sets, and header order/casing, which capture “this request does not fit any known endpoint schema”.
- **Cheap text baselines:** character n -gram TF-IDF is essentially free and, as we show in Section 6.5, actually *outperforms* the embedding on CSIC—so it is at minimum a mandatory control, and potentially a component to fuse with the embedding on harder datasets.
- **Rate / session features** (requests per client per second) to catch brute-forcing and scanning, which are invisible at the single-request level.
- **A domain-pretrained language model:** pretraining on large corpora of HTTP/URLs before fine-tuning on CSIC would likely improve the embedding, especially in the low-data regime we used.

A word of caution consistent with Claim 4: features such as the raw `\r\n` terminator or fixed `Accept` headers *would* boost CSIC / IDS2018 scores but only by exploiting generation artifacts; they should be avoided, not added.

5 Reproducibility Analysis

5.1 Can the code be executed?

Yes. Because we could not locate an official public code release from the authors, we *re-implemented* the full HTTP2vec pipeline as a small, modular Python package (`src/http2vec`) and drove it from a Jupyter notebook. The notebook ran end-to-end on a Google Colab GPU (CUDA): it clones the repository, installs the package, downloads CSIC 2010, trains the tokenizer and RoBERTa MLM, computes embeddings, trains all classifiers, and writes every figure and metric table. All results in this report come from that single executed run.

5.2 Are all required files and dependencies available?

Dependencies are declared in `pyproject.toml` / `requirements.txt` (`torch`, `transformers`, `tokenizers`, `scikit-learn`, `numpy`, `pandas`, `matplotlib`, `seaborn`). The version constraints are conservative *floors* rather than exact pins, so a future incompatible release could in principle break the run; pinning the exact `pip freeze` would make it byte-for-byte reproducible. The CSIC 2010 raw files are deliberately *not* committed (they contain real attack payloads); the repository provides a download helper and clear instructions, and the loader raises an explicit error listing the missing files if they are absent. Determinism is supported: a fixed seed (42) is threaded through data subsetting, the train/test splits, the MLM trainer and the MLP head.

5.3 Are there hidden preprocessing steps?

Nothing is hidden—the whole pipeline is explicit in the configuration object—but two preprocessing choices are subtle and materially affect results:

1. **CR/LF handling.** Our parser treats CR and LF strictly as line separators and strips a trailing `\r` from every line. This has an important side effect: it *removes* the “anomalies end in `\n`, not `\r\n`” artifact that the paper identified as making CSIC easy. Our reproduction is thus a *cleaner, harder* test than the original, which is one plausible reason our numbers are lower (Section 6.8).
2. **Subset and epochs (a hardware-imposed limitation).** The executed run used a seeded **40% subset** of each file and **5 epochs** (vs the paper’s full data and 10 epochs). This was *not* a modelling preference but the practical ceiling of our environment: the whole pipeline was run in a single **Google Colab** session on a **T4 GPU (16 GB)**, where it took almost **4 hours** end-to-end and peaked at **15.5 of the 16 GB** of GPU memory. Larger fractions or more epochs would have exceeded either the session time limit or the T4’s memory, so 40%/5 epochs were the largest settings that completed reliably. This is a documented, intentional scaling-down—not a hidden step—but it is a genuine limitation of *our* reproduction (distinct from the paper’s own limitations) and must be kept in mind when comparing to the paper, whose full run used two RTX 2080Ti GPUs for about 7 hours.

5.4 Overall reproducibility

The paper is reproducible in spirit but not to the digit. We reproduced its central qualitative claim with an independent implementation, which is the strong form of reproducibility. Exact numeric reproduction is hindered by: the absence of official code; sensitivity to language-model training details (epochs, data size, library versions, mixed precision); the high variance of tail metrics such as FPR_{99} ; and the artifact-handling differences above. The paper also reports a heavy cost (about 7 hours on two RTX 2080Ti GPUs for CSIC 2010), which is itself a reproducibility barrier, and it candidly documents a failed reproduction of the CAE baseline it compares against—a useful honesty signal, but also a reminder that “irreproducible without the original source code” is a real risk in this literature.

6 Experimental Results

6.1 Experimental setup and the modifications we introduced

Our reproduction reuses the paper’s architecture and protocol but adds several experiments to stress-test the claims. The run reported here used a seeded 40% subset of CSIC 2010, a RoBERTa MLM (6 layers, hidden 768, 12 heads, intermediate 3072, vocab 30,000, max length 512) trained for 5 epochs (batch 32, learning rate 5×10^{-5} , 15% masking, weight decay 0.01, warmup 0.06, mixed precision on GPU), and 3072-dimensional embeddings (concatenation of the last four hidden layers, mean-pooled). **Compute environment.** Everything ran in one Google Colab

session on a single **T4 GPU (16 GB)**, taking almost **4 hours** end-to-end and using up to **15.5 GB** of the 16 GB of GPU memory; the 40%/5-epoch budget was dictated by this ceiling rather than chosen for modelling reasons (see the limitation noted in Section 5). Beyond the paper we:

- added two more supervised classifiers—**gradient boosting** and **KNN**—to the paper’s LR/R-F/SVC, to see whether performance is specific to the paper’s three models;
- added a second unsupervised detector, **Local Outlier Factor**, alongside an **Isolation Forest**, to test the paper’s unsupervised speculation;
- added a small trainable **MLP head** over the frozen embeddings (a “fine-tune-like” extension);
- introduced a single **shared, seeded, stratified holdout** (70/30) so that every model—supervised, unsupervised and the MLP—is scored on *exactly the same* test set for a fair head-to-head comparison (the paper reports CV for its three models only);
- added two **non-embedding baselines**—hand-crafted descriptive features and a character n -gram TF-IDF model—as the controls the paper omits, to test whether the embedding is even necessary (Section 6.5);
- added a **duplicate / leakage / de-duplication** analysis to quantify how much CSIC’s templating inflates the numbers (Sections 3.1, 6.9);
- added a **PCA** analysis of the embedding space (Section 4).

We report both the paper’s protocol (stratified 5-fold CV) and the shared holdout.

6.2 The RoBERTa language model

Figure 4 shows the MLM learning curve. The training loss falls sharply from ≈ 10.3 to ≈ 1.0 and then plateaus; the per-epoch validation loss (on a seeded 5% held-out slice of *normal* traffic) tracks it down to ≈ 0.9 and never turns back up. This indicates the model learned the structure of normal HTTP without obvious overfitting, even in the reduced 5-epoch budget. The corresponding perplexity (e^{loss}) drops by orders of magnitude, confirming the language model is genuinely predictive of masked tokens in normal traffic.

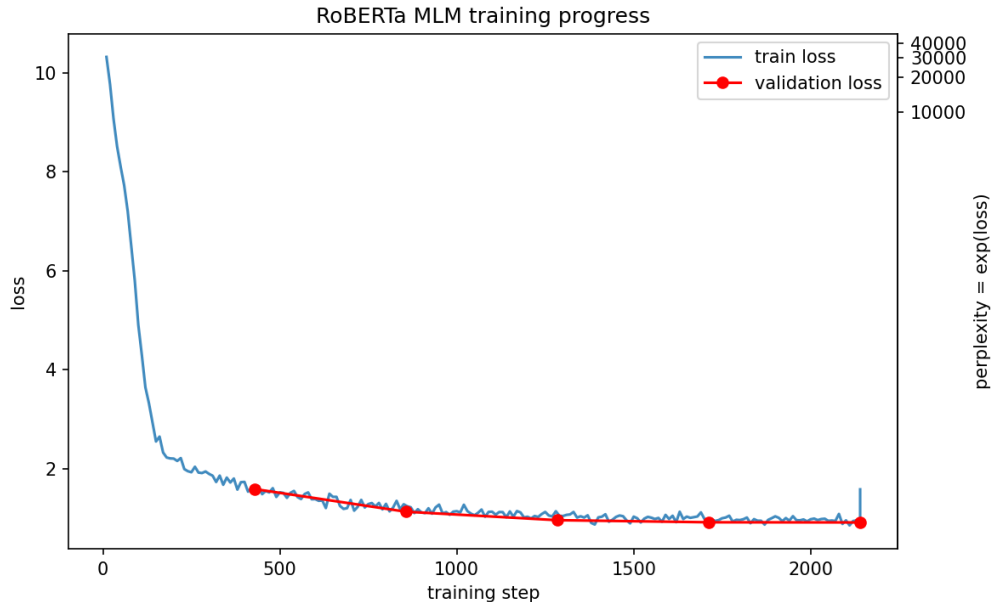


Figure 4: RoBERTa MLM training progress: per-step training loss (blue) and per-epoch validation loss (red), with a perplexity axis on the right. Steady convergence with the validation loss tracking training loss indicates learning without overfitting.

6.3 Evaluation metrics: definitions and why they matter here

We choose metrics deliberately for an imbalanced, cost-asymmetric security task rather than reporting a fixed list. Let TP, FP, TN, FN be the confusion-matrix counts with the *anomaly* as the positive class.

- **Precision** = $\frac{TP}{TP + FP}$: of the requests we flag as attacks, how many really are. Low precision $\Rightarrow$ many false alarms $\Rightarrow$ analyst fatigue and eventual alert-muting.
- **Recall (TPR)** = $\frac{TP}{TP + FN}$: of the real attacks, how many we catch. Low recall $\Rightarrow$ missed attacks $\Rightarrow$ breaches. This is usually the costlier error in an IDS.
- **F1** = $\frac{2PR}{P + R}$: harmonic mean of precision and recall; a single number that punishes ignoring either.
- **F $_{\beta}$** = $\frac{(1 + \beta^2)PR}{\beta^2P + R}$ with $\beta = 2$: weights recall four times as heavily as precision, encoding “missing an attack is worse than a false alarm”.
- **MCC** = $\frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$: a balanced correlation between predictions and truth that stays informative under class imbalance; our preferred single summary.
- **ROC-AUC**: the area under the TPR-vs-FPR curve; a threshold-independent measure of how well the anomaly *score* ranks attacks above normals.
- **FPR₉₀, FPR₉₉**: the false-positive rate incurred to reach 90% / 99% recall—the paper’s headline and the most *operational* metric, because it answers “how much benign traffic must I block to catch (almost) all attacks?”
- **Accuracy**: reported for completeness but de-emphasised, since at the unrealistic 41% base rate it flatters the models (Section 3.2).

The FP/FN asymmetry in this domain. A false negative is a *successful attack that went undetected* (SQL injection, XSS)—potentially a data breach. A false positive is a *blocked or alerted legitimate request*—annoying, and in volume corrosive to trust in the system. Because attacks are rare in reality, even a small FPR translates into a flood of false alarms, so we watch FPR₉₉ closely; because a missed attack is expensive, we weight recall via $\beta = 2$. This is why we regard ROC-AUC, MCC and FPR@TPR—not accuracy—as the metrics that decide whether the method is deployable.

6.4 Supervised classifiers and the unified comparison

Table 3 is the central comparison of the *embedding-based* models: every model scored on the same held-out test set (7,328 requests). Among these, the paper’s linear models dominate—**Logistic Regression** is best (F1 = 0.913, MCC = 0.853, ROC-AUC = 0.984), with linear SVC a close second, exactly matching the paper’s finding that the embedding space is close to linearly separable. The tree ensembles (RF, gradient boosting) and KNN are clearly weaker on this high-dimensional representation. Our added **MLP head** is competitive (F1 = 0.891) but does not beat plain logistic regression, a nice illustration that a linear model on a good representation is hard to beat. The two *unsupervised* detectors are far behind (Isolation Forest F1 = 0.473; LOF F1 = 0.336)—the quantitative core of our rebuttal to the paper’s unsupervised speculation. The 5-fold cross-validated numbers (paper protocol) are within ± 0.01 of the holdout for the supervised models, so the ranking is stable. Crucially, this table shows only how the embedding models rank *among themselves*; Section 6.5 asks the more pointed question of whether the embedding is needed *at all*.

Model	F1	F ₂	MCC	AUC	Prec.	Rec.	Acc.	FPR ₉₀	FPR ₉₉
Logistic Regression	0.913	0.910	0.853	0.984	0.918	0.909	0.929	0.050	0.210
Linear SVC	0.910	0.908	0.847	0.980	0.913	0.907	0.926	0.056	0.282
MLP head (ours)	0.891	0.891	0.816	0.979	0.893	0.890	0.911	0.080	0.219
Gradient Boosting (ours)	0.862	0.857	0.767	0.964	0.870	0.854	0.888	0.127	0.304
Random Forest	0.836	0.841	0.720	0.950	0.827	0.844	0.864	0.162	0.345
KNN (ours)	0.798	0.759	0.683	0.945	0.873	0.735	0.847	0.194	0.344
Isolation Forest*	0.473	0.414	0.250	0.750	0.622	0.382	0.651	0.477	0.768
Local Outlier Factor*	0.336	0.240	0.360	0.837	1.000	0.202	0.672	0.649	0.953

Table 3: Unified comparison of the *embedding-based* models on the shared holdout test set ($n = 7,328$). * Unsupervised detectors fit on normal-only embeddings. Best value per column in bold. “(ours)” marks models added beyond the paper. Non-embedding baselines are compared separately in Table 4.

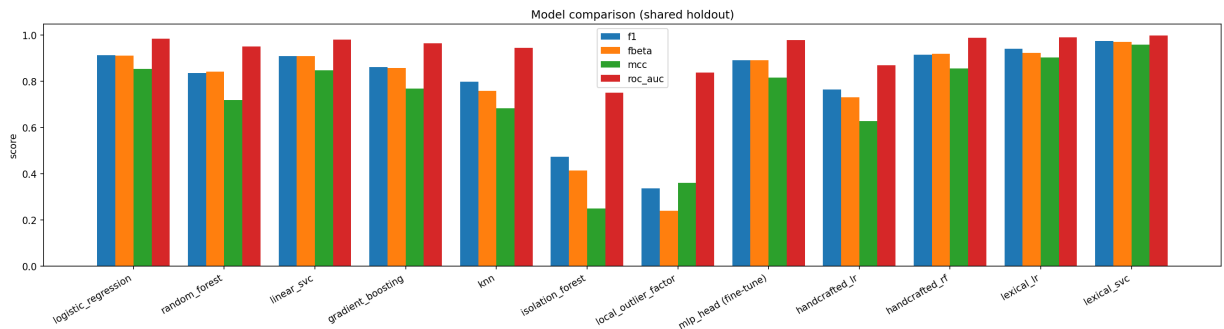


Figure 5: F1, F₂, MCC and ROC-AUC per model on the shared holdout. The supervised linear models and the MLP head cluster at the top; the unsupervised detectors (right) collapse on the threshold metrics while retaining moderate ROC-AUC.

6.5 Do we even need the embedding? Hand-crafted and lexical baselines

The paper’s central premise is that a *learned* contextual representation is worth its cost. The obvious control—does it beat cheap, classic features?—is absent from the paper, so we added it. On the *same* shared holdout we trained two non-embedding baselines: (i) a **hand-crafted** baseline (Logistic Regression and Random Forest on the 16 descriptive features of Section 3), and (ii) a **lexical** baseline (a character n -gram TF-IDF representation, **char_wb**, $n \in [2, 5]$, fed to Logistic Regression and a linear SVC). Table 4 and Figure 6 report the result, and it is striking.

Model	F1	F ₂	MCC	AUC	FPR ₉₀	FPR ₉₉
RoBERTa embedding + LR (best embedding)	0.913	0.910	0.853	0.984	0.050	0.210
Hand-crafted features + LR	0.765	0.730	0.628	0.869	0.341	0.896
Hand-crafted features + RF	0.915	0.919	0.856	0.988	0.053	0.125
Char n -gram TF-IDF + LR	0.941	0.922	0.903	0.991	0.014	0.206
Char n -gram TF-IDF + SVC	0.975	0.969	0.958	0.998	0.001	0.034

Table 4: The best embedding model versus non-embedding baselines on the same shared holdout ($n = 7,328$). Best value per column in bold. The lexical char n -gram SVC dominates the RoBERTa embedding on *every* metric, and a hand-crafted Random Forest matches it.

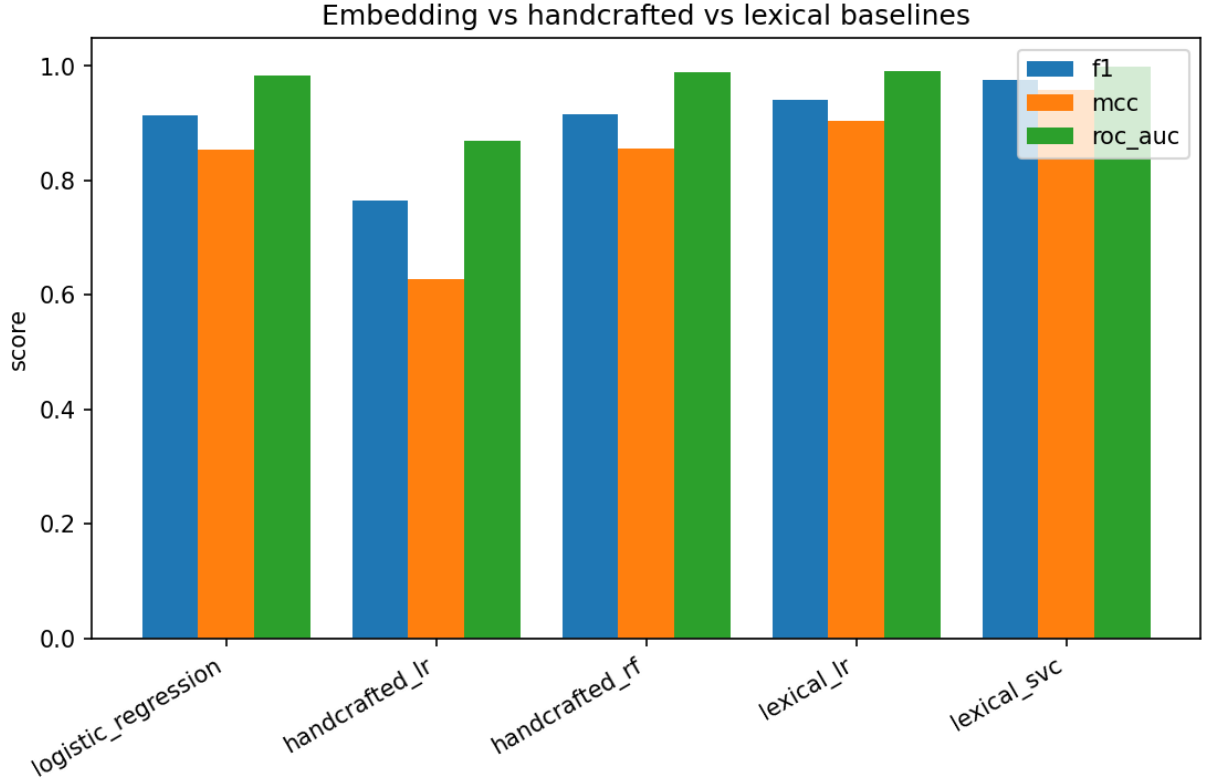


Figure 6: Best embedding model vs hand-crafted and lexical baselines (F1, MCC, ROC-AUC). The character n -gram TF-IDF SVC (rightmost) is the best model overall.

The embedding is not necessary on CSIC 2010. The character n -gram SVC reaches $F1 = 0.975$, $MCC = 0.958$ and—most importantly— $FPR_{99} = 0.034$, i.e. an order of magnitude better on the operational tail metric than the embedding’s 0.210, while costing seconds to train and requiring no GPU. Even the hand-crafted Random Forest ($F1 = 0.915$, $FPR_{99} = 0.125$) matches or beats the embedding. Only the hand-crafted *linear* model is genuinely weak ($F1 = 0.765$), which simply says the descriptive features are not linearly separable, not that they lack signal (the RF unlocks them). **Interpretation.** This is not evidence that the embedding is bad—it is strong evidence that CSIC 2010 is *too easy and too lexical* to justify a contextual language model: attacks differ from normal traffic in exactly the surface character patterns (percent-encoding, SQL/XSS keywords, unusual symbols) that a char n -gram model captures directly. It also reframes the paper’s “SOTA” claim (Claim 1): a method is not state-of-the-art on a benchmark where a bag of character n -grams beats it. The honest conclusion is that the embedding’s value must be demonstrated on harder, less templated traffic than CSIC—not on the dataset where the cheapest possible baseline already wins. Figure 7 is the 2D t-SNE view referenced in Claim 3. There is genuine structure—a large, near-pure normal cluster and many satellite clusters by request type—but several satellites mix red and blue, so 2D separation is imperfect even though the full-dimensional classifiers separate the classes well.

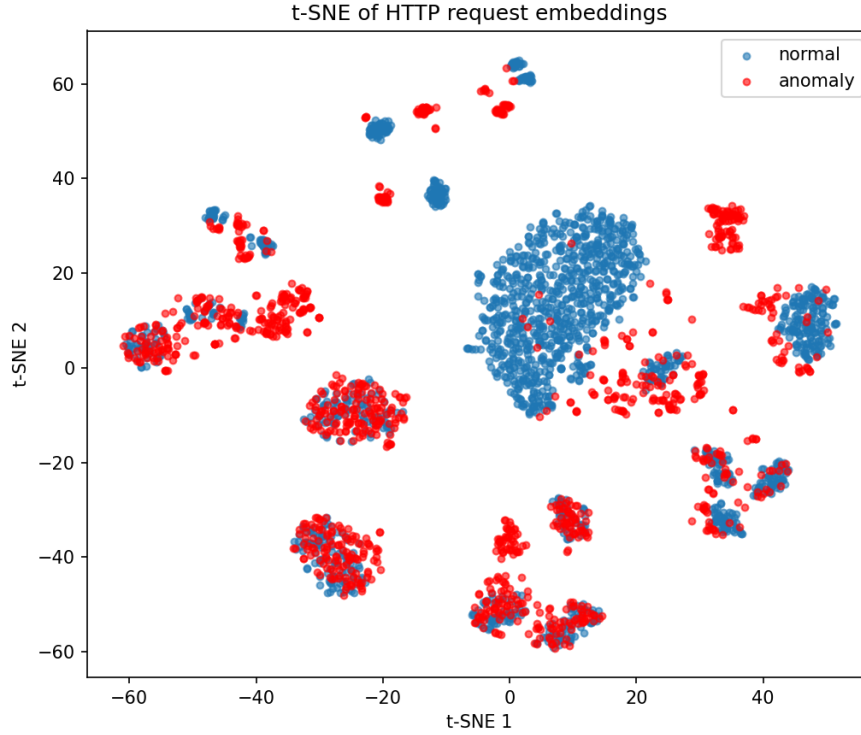
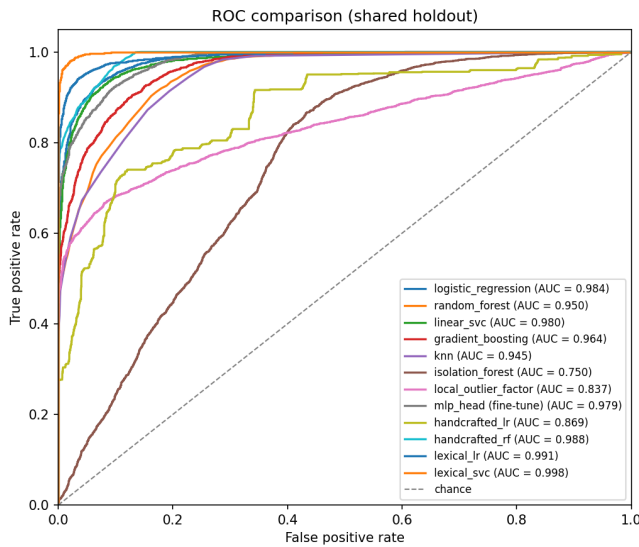


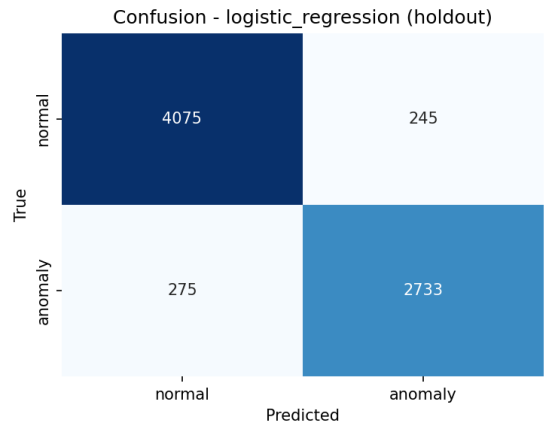
Figure 7: t-SNE of a 3,000-request sample of the embeddings. Clear clustering by request type, one dominant normal cluster, but several clusters are class-mixed—so the “well separated in 2D” claim is only partially supported.

6.6 Ranking quality and the best model

The ROC overlay (Figure 8a) makes the ranking gap explicit: LR, linear SVC and the MLP head sit on top ($\text{AUC} \approx 0.98$), the tree/KNN models a step below, and the unsupervised detectors far lower (Isolation Forest $\text{AUC} = 0.75$). The best supervised model’s confusion matrix (Figure 8b) shows a well-balanced error profile—245 false positives vs 275 false negatives—consistent with its high MCC.



(a) ROC of all models (shared holdout).



(b) Confusion matrix, Logistic Regression.

Figure 8: Threshold-free ranking (left) and the best supervised model’s error profile (right).

6.7 The trainable MLP head

Figure 9 shows the MLP head’s learning curve: early stopping halted training after 19 epochs at the best validation loss (0.179). The training loss keeps falling while validation loss flattens with mild noise—the expected mild-overfitting signature that early stopping is designed to catch. The head performs on par with the linear models but does not exceed them, reinforcing that the representation, not the classifier, is doing the heavy lifting.

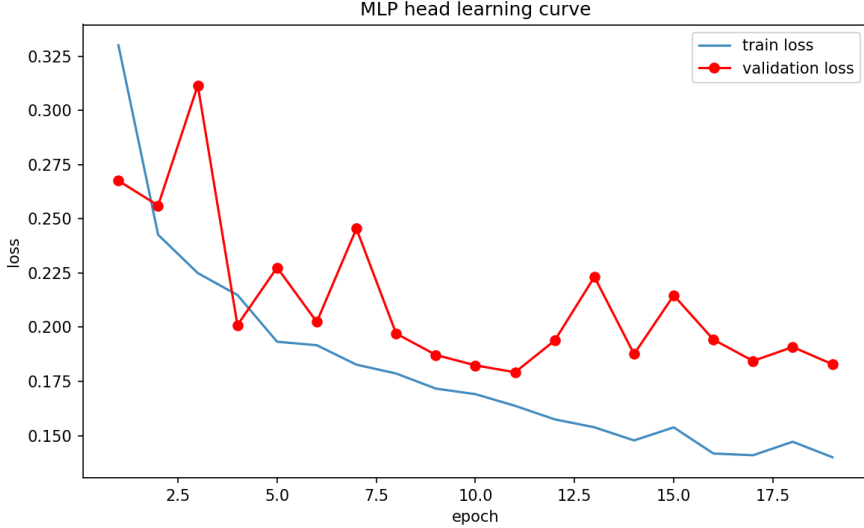


Figure 9: MLP head learning curve (train vs validation loss). Early stopping restores the best-validation weights; the head matches but does not beat logistic regression.

6.8 Reproduction vs the paper

Table 5 compares our 5-fold CV numbers on CSIC 2010 with the paper’s full reported results (both use stratified 5-fold CV, so this is a fair comparison), now across all four metrics the paper reports (F1, MCC, FPR_{90} , FPR_{99}). The picture is consistent: we *reproduce the qualitative claim* (strong F1/MCC from simple classifiers on the embeddings) but land a few F1/MCC points lower and, on the operational tail metrics, *much* higher—our FPR_{99} is 0.23 vs the paper’s 0.075 for LR, and 0.30 vs 0.042 for SVC (a 3–7 $\times$ gap). Our ranking also differs: the paper’s best is SVC and its RF is excellent ($\text{F1} = 0.959$), whereas our best is LR and our RF is markedly weaker (0.833). For context, the paper’s other two datasets are UMP ($\text{F1} \approx 0.926$, $\text{FPR}_{99} \approx 0.15$ for all three classifiers) and CSE-CIC-IDS2018 ($\text{F1} = 0.999$, $\text{FPR}_{99} = 0.000$)—the latter a near-perfect score the authors themselves attribute to a header artifact (Claim 4), so it is not a meaningful target to reproduce.

Classifier	F1		MCC		FPR_{90}		FPR_{99}	
	Paper	Repro	Paper	Repro	Paper	Repro	Paper	Repro
Logistic Regression	0.951	0.916	0.916	0.858	0.017	0.048	0.075	0.227
Linear SVC	0.969	0.909	0.948	0.845	0.003	0.058	0.042	0.299
Random Forest	0.959	0.833	0.930	0.714	0.010	0.169	0.070	0.333

Table 5: Paper vs our reproduction on CSIC 2010 (stratified 5-fold CV), across all four metrics the paper reports. We reproduce the qualitative result but not the paper’s low false-positive rates or its RF strength.

Why the gap? Several factors plausibly combine: (i) our reduced budget (40% data, 5 vs 10 epochs) yields a weaker language model and thus weaker embeddings; (ii) our parser *re-*

moves the `\r\n` artifact the paper exploited, making our test genuinely harder (and arguably more honest); (iii) implementation differences (library versions, a histogram gradient-booster, `max_iter` settings); and (iv) tail metrics like FPR_{99} are inherently high-variance and degrade quickly when the embedding is even slightly weaker. The direction of every one of these is “our setup is harder/smaller”, which is exactly consistent with reproducing the qualitative claim while under-shooting the exact numbers.

6.9 The cost of templating: leakage and de-duplicated re-evaluation

Section 3.1 showed that 75.3% of requests reuse one of only 6,040 request-line templates. Here we quantify what that does to the reported numbers. First, **leakage across the split**: of the 7,328 held-out test requests, 5,691 (**77.7%**) reuse a request-line template that also appears in the training split. A model can lean on template identity rather than on genuinely generalising. Second, a **de-duplicated re-evaluation**: we keep one representative per request-line template (keep-first, reducing $24,426 \rightarrow 6,040$ rows, so no template can span the train/test split) and refit the best model (Logistic Regression). The effect is dramatic (Table 6): F1 slips modestly ($0.913 \rightarrow 0.883$), but MCC drops from 0.853 to 0.649 and the operational FPR_{99} *explodes* from 0.210 to 0.796. In other words, once template repetition is removed, “catch 99% of attacks” costs almost *four* out of five benign requests as false alarms—the model was leaning heavily on seen templates.

Logistic Regression	Full data	De-duplicated	Δ
F1	0.913	0.883	−0.030
MCC	0.853	0.649	−0.204
FPR_{99}	0.210	0.796	+0.586

Table 6: Best model re-evaluated after keep-first de-duplication by request-line template ($24,426 \rightarrow 6,040$ rows). The two holdouts differ in size, so this is read as a “harder, more honest” test rather than a like-for-like one; the collapse of FPR_{99} shows how much CSIC’s templating flatters the headline numbers.

A note on the unsupervised detectors (verified leakage-free). Because the Isolation Forest and LOF are fit on the LM-training normals and scored on the inference set, we explicitly checked for cross-file leakage: *zero* of the 24,426 inference requests are byte-for-byte duplicates of any LM-training request. So the detectors never score a request they were trained on; their weakness (Table 3) is genuine, not a leakage artifact. This turns a plausible worry into a documented, verified result.

7 Error Analysis

Because the unsupervised Isolation Forest scores the full inference set in order, we can map its errors straight back to the original requests. Its confusion matrix on all 24,426 requests is $\text{TN} = 12,109$, $\text{FP} = 2,291$, $\text{FN} = 6,309$, $\text{TP} = 3,717$: it misses roughly two-thirds of attacks (recall 0.37). The score distributions (Figure 10, left) explain why—the normal and anomaly score densities overlap heavily, with only the normal mode cleanly resolved—so no threshold separates them well. This is the visual counterpart of the low unsupervised numbers in Table 3.

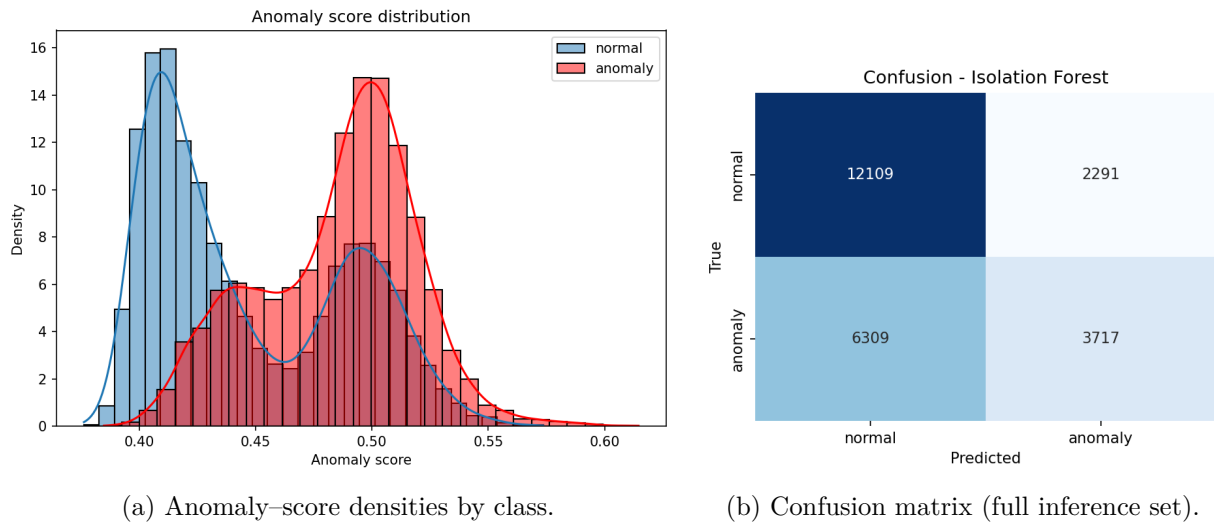


Figure 10: Isolation Forest error analysis. The heavy overlap of the score distributions (left) yields the poor confusion matrix (right): most attacks are missed.

7.1 Examples of failures — unsupervised Isolation Forest

False positives (benign flagged as attack). The flagged-benign requests are legitimate, heavily parameterised e-commerce requests with percent-encoded accented Spanish characters:

```
GET /tienda1/publico/anadir.jsp?id=1&nombre=Jam%F3n+Ib%E9rico&precio=39&cantidad=41&B1=A%F1adir+
al+carrito
POST /tienda1/publico/anadir.jsp
POST /tienda1/publico/caracteristicas.jsp
```

Their percent-encoding (%F3, %E9, %F1) is benign but rare, so a detector that learned “normal traffic has little encoding” treats them as outliers.

False negatives (attacks missed). The missed attacks structurally resemble ordinary login / parameter traffic:

```
GET /publico/autenticar.jsp?modo=entrar&login=bob%40%3CScRipt%3Ealert%28Paros%29%3C%2FscrIPT%3E
...&pwd=...
GET /publico/autenticar.jsp?modo=entrar&login=grimshaw&pwd=84m3ri156&rememberA=on&B1=Entrar
GET /publico/entrar.jsp?errorMsg=%2B
```

The first hides an XSS payload inside a `login` field; the others are near-normal in shape. Because the request skeleton matches high-density normal regions, the density-based detector places them among inliers.

7.2 Examples of failures — best supervised model (Logistic Regression)

The supervised model is the one we would actually deploy, so its mistakes matter most. On the shared holdout it makes 245 false positives and 275 false negatives—a far smaller and more balanced error set than the Isolation Forest. Mapping them back to the original requests reveals a clear pattern: nearly all of them are `POST` requests to the *same handful of templates* that also dominate normal traffic, where the discriminating content lives in the request *body* rather than the first line.

False positives (benign flagged as attack).

```
POST /tienda1/miembros/editar.jsp
POST /tienda1/miembros/editar.jsp
POST /tienda1/publico/autenticar.jsp
```

False negatives (attacks missed).

```
GET /tienda1/publico/registro.jsp?modo=registro&login=thi-cuc&password=PRof4rTo&nombre=Edberto&
apellidos=Genov%E9s&email=keres_murriata%40oleoschamizo.in&dni=78789679M&direccion=Carrer+
Canigo+163%2C+13%3FG&ciudad=Camargo&cp=26325&provincia=Almer%EDa&ntc=0053814597179306&B1A=
Registrar
POST /tienda1/miembros/editar.jsp
POST /tienda1/publico/autenticar.jsp
```

The very same endpoints (`editar.jsp`, `autenticar.jsp`) appear in *both* lists: the model cannot separate normal from attack on these templates because the malicious payload sits in the POST body, which barely moves the mean-pooled embedding. The one GET false negative (`registro.jsp`) is a long but superficially well-formed registration whose anomaly is subtle. Contrast this with the unsupervised detector, whose errors were dominated by *rare encodings* (false positives) and *low-footprint XSS* (false negatives): labels teach the supervised model which rare patterns are actually malicious, shrinking and balancing the error set—but they do not help where the signal is simply not present in the pooled representation.

7.3 Patterns, implications and the FP/FN trade-off

Pattern. The unsupervised detector errs on *rare-but-benign encodings* (false positives) and on *low-footprint attacks that mimic normal structure* (false negatives). The best *supervised* model, by contrast, has a balanced and much smaller error set (245 FP vs 275 FN, Figure 8b and Section 7.2), because labels teach it which rare patterns are actually malicious. **Cybersecurity implication.** The unsupervised false negatives include a real XSS attempt—exactly the failure that matters, since a missed XSS or SQL injection is a potential breach; the false positives, while less dangerous, would (at scale) erode operator trust and cause genuine alerts to be ignored. **Trade-off.** The decision threshold on the continuous anomaly score is the single knob that trades these off: lowering it raises recall (fewer missed attacks) at the cost of more false alarms. This is precisely why we report FPR_{90}/FPR_{99} and weight recall with $\beta = 2$ —in an IDS we accept extra false alarms to avoid missing attacks, but the reproduced $FPR_{99} \approx 0.21$ (rising to ≈ 0.80 once template leakage is removed, Section 6.9) shows that “catch 99% of attacks” still costs an operationally heavy false-alarm rate on this representation.

8 Conclusions

8.1 Claim verdicts at a glance

Table 7 summarises the critical evaluation of Section 2 against our evidence.

Author’s claim	Verdict	Evidence from our reproduction
RoBERTa embeddings give SOTA <i>unsupervised</i> detection	Reproduced, but <i>not</i> SOTA	Reproduced F1 = 0.91, ROC-AUC = 0.98; but a char n -gram TF-IDF SVC beats it (F1 = 0.975, FPR ₉₉ = 0.034; Table 4), and FPR ₉₉ is far higher than the paper.
Representation is unsupervised / suits label-free use	Not supported	Isolation Forest F1 = 0.45, LOF F1 = 0.34 on the same embeddings.
Classes “well separated” in 2D (t-SNE)	Partially supported	Clear clustering, but many satellite clusters are class-mixed (Fig. 7).
Near-perfect scores on CSE-CIC-IDS2018	Artifact, not method quality	The authors’ own analysis attributes it to fixed Accept headers (leakage).
Method is interpretable	Plausible (not tested)	Out of scope here; the token-removal technique is sound but costly.
Evaluation methodology is appropriate	Mostly, with caveats	Good CV/metrics/leakage control; but unrealistic 41% base rate and dataset artifacts inflate results.

Table 7: Verdicts on the paper’s main claims.

8.2 Key findings

- The paper’s core idea works: a RoBERTa MLM trained on normal HTTP traffic produces embeddings on which *simple, linear* supervised classifiers achieve strong detection (F1 = 0.91, MCC = 0.85, ROC-AUC = 0.98), even at 40% data and 5 epochs.
- **But the embedding is not necessary—or SOTA—on CSIC 2010:** a trivial character n -gram TF-IDF + SVC baseline *beats* it on every metric (F1 = 0.975, MCC = 0.958, FPR₉₉ = 0.034 vs the embedding’s 0.913/0.853/0.210), and a hand-crafted Random Forest matches it. The dataset is too lexical and too easy to justify a contextual language model.
- Unsupervised detection on those same embeddings is weak (F1 = 0.34–0.47), contradicting the paper’s optimistic outlook; we verified the detectors are leakage-free (0% exact overlap with their training normals).
- The embedding is highly redundant—~14 PCA components capture 90% of variance—so the paper’s “too large” limitation is real and fixable.
- Much of CSIC 2010’s apparent easiness comes from artifacts (CRLF terminators, PUT-only-in-anomalies) and templating: 75.3% of requests reuse one of only 6,040 request-line templates, 77.7% of test requests share a template with the train split, and de-duplicating by template collapses the best model’s FPR₉₉ from 0.21 to 0.80. The 41% attack prevalence is also unrealistically high.

8.3 Lessons learned

On a templated, lexical benchmark like CSIC 2010 the expensive representation is not what matters—a bag of character n -grams beats the transformer, and the task’s apparent difficulty is dominated by base rate and template leakage rather than by any subtle structure the embedding could capture. More generally: a method is only “state-of-the-art” relative to the baselines it is actually compared against, so the omission of a cheap lexical control is a serious gap; and critically evaluating a security paper means interrogating the *data and metrics*—prevalence, duplication, leakage—at least as hard as the model. The most impressive numbers in this literature are often the least trustworthy.

8.4 Strengths and weaknesses of the proposed solution

Strengths: no hand-crafted features or OOV problem; trains on normal traffic only; strong, near-linear separability; genuine interpretability mechanism; honest reporting of artifacts. **Weak-**

nesses: very large (~ 3000 -d) and redundant representation; heavy compute (hours on multiple GPUs); detection is really supervised despite the “unsupervised representation” framing; *no cheap baseline is reported, and when we add one a character n -gram model beats the embedding on CSIC*, so the benefit over free lexical features is unproven on this benchmark; results validated only on artifact-laden benchmarks at unrealistic base rates; no temporal / drift evaluation.

8.5 Suggestions for future improvements

Compress the embedding with PCA/agglomeration ($3072 \rightarrow \sim 50$) to cut cost with negligible accuracy loss; evaluate at realistic ($\ll 1\%$) attack base rates and report precision/PR-AUC there; add response-side and session/rate features; test true generalisation on modern, non-artifactual traffic and across time; and, if unsupervised detection is the goal, pursue density models in the *reduced* space or one-class objectives rather than assuming the supervised embedding transfers.

9 Executive Summary

Problem and source. We reproduced and critically evaluated *HTTP2vec* (Gniewkowski et al., 2021), which tackles web-attack detection by treating each HTTP request as text. A byte-level BPE tokenizer and a RoBERTa masked-language model are trained on *normal* traffic; every request is embedded as the mean of its last four hidden layers (3072-d); and simple classifiers (Logistic Regression, Random Forest, linear SVC) label the embeddings as normal or anomalous. The paper reports up to $F1 = 0.969$ on CSIC 2010 and a “perfect” $F1 = 0.999$ on CSE-CIC-IDS2018, and highlights interpretability and clustering as key benefits.

What we did. We built an independent, modular re-implementation of the whole pipeline and ran it end-to-end on CSIC 2010 (a seeded 40% subset, 5 epochs, single GPU). Beyond reproducing the paper’s three classifiers, we added gradient boosting, KNN, a second unsupervised detector (LOF alongside Isolation Forest), a trainable MLP head, and two *non-embedding baselines* (hand-crafted features and a character n -gram TF-IDF model), and we scored *all* models on one shared, seeded, stratified holdout. We performed full EDA (class balance, distributions, Spearman correlation, a three-level duplicate analysis and crosstab), a feature-engineering analysis (encoding, pooling, scaling, PCA), a template leakage / de-duplication study, and an error analysis of both the supervised and the unsupervised model.

Headline results. The paper’s central idea holds *qualitatively*: on the embeddings, Logistic Regression is best with $F1 = 0.913$, $MCC = 0.853$ and $ROC-AUC = 0.984$, with linear SVC and our MLP head close behind—confirming the space is nearly linearly separable. *But the embedding is neither SOTA nor necessary here*: a trivial character n -gram TF-IDF + SVC baseline beats it on every metric ($F1 = 0.975$, $MCC = 0.958$, $FPR_{99} = 0.034$), and a hand-crafted Random Forest matches it. We also did not reproduce the paper’s low false-positive rates ($FPR_{99} \approx 0.21$ for us vs 0.075 reported), and the two *unsupervised* detectors were weak ($F1 = 0.47$ and 0.34), directly contradicting the paper’s speculation that unsupervised detection on these embeddings “is likely to be effective”. PCA showed the 3072-d representation is highly redundant (~ 14 components reach 90% variance).

Critical assessment. Several of the paper’s most striking results are driven by the *data*, not the method: the “perfect” IDS2018 score is a header-leakage artifact (the authors themselves show this); CSIC 2010 contains CRLF-terminator and HTTP-method giveaways, and is so heavily templated that 75.3% of requests reuse one of only 6,040 request-line templates—77.7% of test requests share a template with the training split, and removing that leakage collapses the best model’s FPR_{99} from 0.21 to 0.80. The 41% attack prevalence is also far above any real deployment, inflating precision, F1 and accuracy. The evaluation protocol is otherwise sound (stratified CV, appropriate metrics, no train/inference leakage on the normal side—including into the unsupervised detectors, which we verified in code at 0%).

Bottom line. HTTP2vec is a reasonable *supervised* feature extractor for HTTP anomaly detection, but on CSIC 2010 it is strictly dominated by a character n -gram baseline that costs seconds to train, so its value must be demonstrated on harder, less templated traffic before the RoBERTa machinery is justified. Given labels, a GPU, dimensionality reduction and careful guarding against dataset artifacts and base-rate mismatch it remains a defensible choice—but its unsupervised promise, its exact SOTA numbers, and its “clean 2D separation” narrative are not supported by our reproduction.

10 Summing It Up

Problem addressed

Detecting anomalous / attack HTTP requests (SQL injection, XSS, CRLF, etc.)—a Web Application Firewall / IDS problem.

Selected source

HTTP2vec: Embedding of HTTP Requests for Detection of Anomalous Traffic, Gniewkowski, Maciejewski, Surmacz, Walentynowicz, arXiv:2108.01763 (2021).

Dataset

CSIC 2010 HTTP dataset (normal training / normal inference / anomalous inference); our run used a seeded 40% subset (14,400 / 14,400 / 10,026).

Methodology

BBPE tokenizer + RoBERTa MLM on normal traffic $\rightarrow$ 3072-d embeddings $\rightarrow$ supervised classifiers and unsupervised detectors, plus non-embedding baselines (hand-crafted features, char n -gram TF-IDF) as controls; independent re-implementation, evaluated with stratified CV and a shared holdout using F1, F_2 , MCC, ROC-AUC and FPR@TPR.

Main findings

Strong supervised detection reproduced ($F1 = 0.91$, $AUC = 0.98$), *but a char n -gram TF-IDF SVC beats the embedding* ($F1 = 0.975$, $FPR_{99} = 0.034$); weak unsupervised detection ($F1 = 0.34$ – 0.47); highly redundant embedding; heavy templating (75% of request lines duplicated, 78% train/test leakage) and an unrealistic base rate.

Were the claims supported?

Partially. Supervised separability: yes, qualitatively. “State-of-the-art”: no—a trivial lexical baseline wins. The unsupervised outlook and clean 2D separation: no. The IDS2018 “perfect” result: an artifact.

Most important insight

In security ML the data and the metrics deserve more scrutiny than the model, and a method is only “SOTA” relative to the baselines it is compared against—here a bag of character n -grams beats the transformer, and the benchmark’s difficulty largely reflects leakage and prevalence, not real-world detection power.

Do we recommend this approach for similar problems?

Only with a cheap lexical baseline run alongside it: on CSIC that baseline wins, so the embedding must justify its cost on harder, less templated traffic. As an off-the-shelf *unsupervised* detector: no.

Final conclusion

HTTP2vec is a sound idea whose core supervised claim we independently reproduce, but whose headline results are optimistic and, on CSIC, are beaten by a trivial character n -gram baseline; a rigorous, baseline-, artifact- and base-rate-aware evaluation is essential before trusting such numbers in production.